ChatGPT

how to process the raw data which is from HCNetSDK

Processing **raw data from the Hikvision HCNetSDK** usually means decoding the **compressed video stream** (H.264 / H.265) coming from the camera into **frames (YUV/RGB)** that your C++ application can process.

Since you mentioned earlier you're building a **C++ app to process frames from a Hikvision IP camera**, the normal pipeline looks like this:

---

1. Data Flow from HCNetSDK

When you start real-time preview, the SDK sends **PS stream packets** to your callback.

Typical flow:

```
IP Camera
   |
   |  H.264/H.265 stream
   ▼
HCNetSDK
   |
   |  Raw stream packets
   ▼
RealDataCallback
   |
   ▼
Decoder (PlayM4 or FFmpeg)
   |
   ▼
Raw Frames (YUV420)
   |
   ▼
Convert to RGB
   |
   ▼
Your processing (OpenCV / AI / storage)
```

---

2. Register the Real-Time Data Callback

You start preview and receive raw stream data.

```cpp
void CALLBACK RealDataCallback(
    LONG lRealHandle,
    DWORD dwDataType,
    BYTE *pBuffer,
    DWORD dwBufSize,
    void* pUser)
{
    if (dwDataType == NET_DVR_SYSHEAD)
    {
        // stream header
    }
    else if (dwDataType == NET_DVR_STREAMDATA)
    {
        // compressed stream (H264/H265)
    }
}
```

Start preview:

```cpp
NET_DVR_RealPlay_V40(userID, &previewInfo, RealDataCallback, NULL);
```

---

3. Decode the Stream (Important)

The stream is **compressed**, so you must decode it.

There are two common ways:

Option A — Use Hikvision decoder (PlayM4)

PlayM4 comes with HCNetSDK.

Steps:

1. Open stream
2. Input stream data

3. Get decoded frame callback

Example:

```cpp
PlayM4_OpenStream(port, pBuffer, dwBufSize, 1024*1024);

PlayM4_SetDecCallBack(port, DecCBFun);

PlayM4_Play(port, NULL);
```

Decoded frame callback:

```cpp
void CALLBACK DecCBFun(
    LONG nPort,
    char *pBuf,
    LONG nSize,
    FRAME_INFO *pFrameInfo,
    void *pUser,
    LONG nReserved2)
{
    if (pFrameInfo->nType == T_YV12)
    {
        // raw YUV frame
    }
}
```

---

4. Convert YUV → RGB (for processing)

Decoded frames are usually **YV12 / YUV420**.

You convert them before processing.

Example with **OpenCV**:

```cpp
cv::Mat yuv(height * 3 / 2, width, CV_8UC1, pBuf);
cv::Mat rgb;
```

```
cv::cvtColor(yuv, rgb, cv::COLOR_YUV2BGR_YV12);
```

Now you have:

```
cv::Mat rgb
```

which you can use for:

- AI inference
- object detection
- saving images
- streaming

---

5. Example Full Processing Pipeline

```
HCNetSDK
   |
RealDataCallback
   |
PlayM4_InputData()
   |
Decoder
   |
DecCBFun
   |
YUV frame
   |
OpenCV convert
   |
cv::Mat
   |
AI / analysis
```

---

6. Typical Things Developers Do With Frames

Once frames are decoded you can:

Task	Tool
Motion detection	OpenCV

Task	Tool
Object detection	YOLO
Save snapshots	OpenCV
Video recording	FFmpeg
AI inference	TensorRT

---

7. Performance Tip (Important)

If you're processing **high FPS cameras**, avoid:

```
YUV -> RGB -> YUV -> RGB
```

Instead process **YUV directly** when possible.

It reduces CPU usage.

---

8. Advanced Architecture (Best for Production)

For stable systems:

```
Camera Thread
      |
      ▼
Decode Thread
      |
      ▼
Frame Queue (Lock-Free)
      |
      ▼
AI Worker Threads
```

This prevents frame drops.

✅ If you want, I can also show you:

- **A full working C++ example of HCNetSDK + PlayM4 + OpenCV**
- **Ultra-low latency architecture for multiple Hikvision cameras**
- **How to decode Hikvision streams using FFmpeg instead of PlayM4 (better for Linux)**

The **FFmpeg method is usually much better for Linux servers like Ubuntu**, which you mentioned you're using.

yes

Since you're building a **C++ application with the Hikvision HCNetSDK**, I'll show you a **complete Linux-friendly pipeline using FFmpeg and OpenCV**.

This method is **better for Ubuntu servers** and production systems than the Hikvision `PlayM4` decoder.

---

1. Overall Architecture

Your program should process the camera stream like this:

```
Hikvision Camera
        |
        |  H264/H265 stream
        ▼
HCNetSDK RealDataCallback
        |
        ▼
FFmpeg Decoder
        |
        ▼
AVFrame (YUV)
        |
        ▼
Convert to RGB
        |
        ▼
OpenCV cv::Mat
```

```
        |
        ▼
AI / Computer Vision / Storage
```

---

2. Start Camera Stream with HCNetSDK

You first connect and start real-time preview.

```cpp
NET_DVR_PREVIEWINFO previewInfo = {0};

previewInfo.lChannel = 1;
previewInfo.dwStreamType = 0;   // main stream
previewInfo.dwLinkMode = 0;     // TCP
previewInfo.bBlocked = 1;

NET_DVR_RealPlay_V40(userID, &previewInfo, RealDataCallback, nullptr);
```

---

3. Receive Raw Stream

The SDK pushes **compressed H264/H265 packets**.

```cpp
void CALLBACK RealDataCallback(
    LONG lRealHandle,
    DWORD dwDataType,
    BYTE* pBuffer,
    DWORD dwBufSize,
    void* pUser)
{
    if (dwDataType == NET_DVR_STREAMDATA)
    {
        decode_with_ffmpeg(pBuffer, dwBufSize);
    }
}
```

---

4. Initialize FFmpeg Decoder

Initialize once when program starts.

```cpp
AVCodec* codec = avcodec_find_decoder(AV_CODEC_ID_H264);
AVCodecContext* codec_ctx = avcodec_alloc_context3(codec);

avcodec_open2(codec_ctx, codec, NULL);

AVFrame* frame = av_frame_alloc();
AVPacket* packet = av_packet_alloc();
```

If your camera outputs **H265**, use:

```cpp
AV_CODEC_ID_HEVC
```

---

5. Decode Stream

Send the packet to FFmpeg.

```cpp
void decode_with_ffmpeg(uint8_t* data, int size)
{
    packet->data = data;
    packet->size = size;

    avcodec_send_packet(codec_ctx, packet);

    while (avcodec_receive_frame(codec_ctx, frame) == 0)
    {
        process_frame(frame);
    }
}
```

---

6. Convert YUV → RGB

Camera frames are usually **YUV420P**.

Use FFmpeg scaling.

```cpp
SwsContext* sws_ctx = sws_getContext(
    width, height, AV_PIX_FMT_YUV420P,
    width, height, AV_PIX_FMT_BGR24,
    SWS_BILINEAR, NULL, NULL, NULL);

uint8_t* rgb_data[1];
int rgb_linesize[1];

rgb_data[0] = (uint8_t*)malloc(width * height * 3);
rgb_linesize[0] = 3 * width;

sws_scale(
    sws_ctx,
    frame->data,
    frame->linesize,
    0,
    height,
    rgb_data,
    rgb_linesize);
```

---

7. Convert to OpenCV Frame

Now create an OpenCV matrix.

```cpp
cv::Mat img(height, width, CV_8UC3, rgb_data[0]);
```

Now you can:

```
AI detection
motion detection
save image
stream to web
```

Example:

```cpp
```

```
cv::imshow("camera", img);
```

---

8. Performance Architecture (Important)

For **real-time systems**, do not decode inside the callback.

Instead use a **queue pipeline**.

```
HCNetSDK Thread
        |
        ▼
Packet Queue
        |
        ▼
Decode Thread
        |
        ▼
Frame Queue
        |
        ▼
AI Workers
```

This prevents:

- dropped frames
- blocking callbacks
- SDK timeout

---

9. Memory Optimization

Avoid frequent allocations.

Bad:

```
malloc per frame ❌
```

Good:

```
reuse frame buffers ✔
```

10. Multi-Camera Architecture

If you process multiple cameras:

```
Camera 1 → Decoder Thread 1
Camera 2 → Decoder Thread 2
Camera 3 → Decoder Thread 3
```

Each camera should have its **own decoder context**.

---

11. Latency Comparison

Method	Latency	Stability
PlayM4	medium	limited Linux support
FFmpeg	low	excellent
GStreamer	low	complex

Recommendation:

Use **FFmpeg + HCNetSDK**.

✅ If you'd like, I can also show you **a full production-level C++ project structure** for Hikvision cameras like this:

```
camera-service/
 ├── camera_manager.cpp
 ├── hikvision_client.cpp
 ├── ffmpeg_decoder.cpp
 ├── frame_queue.hpp
 ├── ai_processor.cpp
 └── main.cpp
```

It's the **architecture used in large surveillance systems handling 100+ cameras**.

yes

I'll show you a **production-level C++ architecture for processing multiple cameras using Hikvision HCNetSDK + FFmpeg + OpenCV**.

This design is used in **AI surveillance systems handling 50–500 cameras**.

---

1. Recommended Project Structure

Plain text

```
camera-service/
│
├── main.cpp
├── camera_manager.hpp
├── camera_manager.cpp
│
├── hikvision_client.hpp
├── hikvision_client.cpp
│
├── ffmpeg_decoder.hpp
├── ffmpeg_decoder.cpp
│
├── frame_queue.hpp
│
├── frame.hpp
│
├── ai_processor.hpp
├── ai_processor.cpp
│
└── CMakeLists.txt
```

Each component has a **single responsibility**.

---

2. Core Data Structure

frame.hpp

```cpp
#pragma once
#include <opencv2/opencv.hpp>

struct Frame
{
    int width;
    int height;
    int64_t timestamp;
    cv::Mat image;
};
```

This is the **standard frame format for your system**.

---

3. Thread-Safe Frame Queue

frame_queue.hpp

```cpp
#pragma once
#include <queue>
#include <mutex>
#include <condition_variable>

template<typename T>
class FrameQueue
{
private:
    std::queue<T> queue;
    std::mutex mutex;
    std::condition_variable cond;

public:

    void push(const T& item)
    {
        std::unique_lock<std::mutex> lock(mutex);
        queue.push(item);
        cond.notify_one();
    }

    T pop()
    {
```

```
        std::unique_lock<std::mutex> lock(mutex);

        cond.wait(lock, [&]{
            return !queue.empty();
        });

        T item = queue.front();
        queue.pop();

        return item;
    }
};
```

This queue **decouples camera, decoding, and AI processing**.

---

4. Hikvision Camera Client

hikvision_client.hpp

‹/› C++

```
class HikvisionClient
{
public:

    bool connect(
        const std::string& ip,
        const std::string& user,
        const std::string& pass);

    void start_stream();

private:

    static void CALLBACK realDataCallback(
        LONG handle,
        DWORD type,
        BYTE* buffer,
        DWORD size,
        void* user);
};
```

This module **receives raw stream packets** from the camera.

---

5. FFmpeg Decoder

ffmpeg_decoder.hpp

```cpp
class FFmpegDecoder
{
public:

    FFmpegDecoder();

    void decode(uint8_t* data, int size);

private:

    AVCodecContext* codec_ctx;
    AVFrame* frame;
    AVPacket* packet;
};
```

Responsibilities:

```
compressed H264/H265
        ↓
decode
        ↓
YUV frame
        ↓
convert to RGB
        ↓
Frame object
```

---

6. AI Processing Engine

ai_processor.cpp

```cpp
void AIProcessor::run()
{
    while(true)
    {
```

```
        Frame frame = frameQueue.pop();

        cv::Mat img = frame.image;

        detect_objects(img);
    }
}
```

This thread performs:

- object detection
- motion detection
- face recognition
- custom AI

---

7. Camera Manager

Handles **multiple cameras**.

camera_manager.cpp

```C++
void CameraManager::addCamera(CameraConfig config)
{
    auto camera = std::make_shared<HikvisionClient>();

    camera->connect(
        config.ip,
        config.user,
        config.password);

    camera->start_stream();

    cameras.push_back(camera);
}
```

---

8. Main Application

main.cpp

```cpp
int main()
{
    CameraManager manager;

    manager.addCamera({
        "192.168.1.10",
        "admin",
        "12345"
    });

    manager.addCamera({
        "192.168.1.11",
        "admin",
        "12345"
    });

    while(true)
        std::this_thread::sleep_for(std::chrono::seconds(1));
}
```

---

9. Multi-Camera Thread Model

Production systems usually use:

```text
Camera 1
   |
   ▼
SDK Callback
   |
   ▼
Packet Queue
   |
   ▼
Decoder Thread
   |
   ▼
Frame Queue
   |
   ▼
AI Workers
```

For **10 cameras**:

```
10 SDK threads
10 decoder threads
4–8 AI workers
```

---

10. High Performance Optimization

Important optimizations:

1️⃣ Use lock-free queues

Example:

- moodycamel queue

2️⃣ Use GPU decoding

With **NVIDIA Video Codec SDK**

This allows:

```
200+ camera streams per server
```

3️⃣ Avoid memory allocation per frame

Bad:

```
malloc every frame
```

Good:

```
memory pool
```

---

11. Expected Performance

Typical server:

```
CPU: 16 cores
RAM: 32GB
GPU: RTX 3060
```

Capacity:

Cameras	Resolution
30	1080p
120	720p

---

12. Integration With Your Backend

Since you mentioned earlier you are building a backend with:

- Axum
- PostgreSQL

Your camera service should run as a **separate microservice**:

```Plain text
camera-service (C++)
        |
        | gRPC / WebSocket
        ▼
backend API (Rust Axum)
        |
        ▼
PostgreSQL
```

✅ If you want, I can also show you **the best architecture for processing 100–1000 Hikvision cameras on one cluster** (this is how large smart-city surveillance systems are built).